DEBEZIUM CDC POC

Oracle CDC Source Preparation for Debezium

Oracle source database preparation guide

1. Objective

This document describes the Oracle source-side preparation for a Change Data Capture (CDC) proof of concept using Debezium.

The source database is Oracle Database Free running in Docker. Debezium will eventually capture committed database changes and publish change events to Kafka, without repeatedly querying the source table to detect changes.

Target architecture

Oracle Database → Oracle LogMiner → Debezium Oracle Source Connector → Kafka

> **Current scope:** Oracle database preparation, source table setup, supplemental logging, and Debezium database-user preparation.

Kafka, Kafka Connect, the Debezium connector configuration, and the destination database are separate stages and have not yet been configured as part of this documented work.

2. Environment

Component	Configuration
Operating system	Windows
Container runtime	Docker Desktop
Oracle container	`oracle-poc`
Oracle image	`container-registry.oracle.com/database/free:latest-lite`
Oracle version	Oracle AI Database 26ai Free, Release 23.26.3.0.0
CDB	`FREE`
Source PDB	`CDCDB`
Existing default PDB	`FREEPDB1`
Source schema	`CDC_ADMIN`
Source table	`CDC_ADMIN.CUSTOMERS`
Debezium database user	`C##DBZUSER`

> **Security note:** Passwords are intentionally excluded from this document. Store credentials securely and use environment variables or a secrets manager for connector configuration.

---

3. Understand the Oracle CDC prerequisites

Two database settings are especially important.

3.1 ARCHIVELOG mode

Oracle records database changes in redo logs. ARCHIVELOG mode allows filled online redo logs to be archived, which supports recovery and log-based CDC operations.

Check the current mode as an administrative user:

```
SELECT LOG_MODE
FROM V$DATABASE;
```

Expected result:

```
LOG_MODE
------------
ARCHIVELOG
```

> **Result in this environment:** `ARCHIVELOG` was already enabled.

We verified the setting; we did not enable ARCHIVELOG during this setup.

If a different environment reports `NOARCHIVELOG`, the DBA must enable it using the appropriate Oracle administrative procedure. This operation may require a database restart and must not be performed casually in production.

3.2 Minimum supplemental logging

Supplemental logging records additional information in Oracle redo records so that a CDC connector can identify changed rows.

Check the current settings:

```
SELECT LOG_MODE, SUPPLEMENTAL_LOG_DATA_MIN
FROM V$DATABASE;
```

Before the change, the minimum supplemental logging setting was:

```
LOG_MODE     SUPPLEMENTAL_LOG_DATA_MIN
------------ ------------------------
ARCHIVELOG   NO
```

We enabled minimum supplemental logging from `CDB$ROOT` using a SYSDBA connection:

```
ALTER DATABASE ADD SUPPLEMENTAL LOG DATA;
```

The command returned:

```
Database altered.
```

We then verified the setting:

```
SELECT LOG_MODE, SUPPLEMENTAL_LOG_DATA_MIN
FROM V$DATABASE;
```

Expected result:

```
LOG_MODE     SUPPLEMENTAL_LOG_DATA_MIN
------------ ------------------------
ARCHIVELOG   YES
```

> **Result in this environment:** Minimum supplemental logging is enabled at the database level.

---

4. Create a dedicated source table

The source table is located in the `CDCDB` pluggable database, under the `CDC_ADMIN` schema.

The table was created with the following structure:

```
CREATE TABLE CUSTOMERS (
    CUSTOMER_ID NUMBER(10) PRIMARY KEY,
    FIRST_NAME  VARCHAR2(100) NOT NULL,
    LAST_NAME   VARCHAR2(100) NOT NULL,
    EMAIL       VARCHAR2(255) UNIQUE,
    PHONE       VARCHAR2(30),
    CREATED_AT  TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
    UPDATED_AT  TIMESTAMP
);
```

4.1 Tablespace preparation

During setup, the `USERS` tablespace was not available in `CDCDB`.

We created it as an administrative user after switching to the PDB:

```
ALTER SESSION SET CONTAINER = CDCDB;

CREATE TABLESPACE USERS
    DATAFILE SIZE 100M
    AUTOEXTEND ON
    NEXT 10M
    MAXSIZE 1G;
```

We configured the schema's default tablespace and quota:

```
ALTER USER CDC_ADMIN DEFAULT TABLESPACE USERS;

ALTER USER CDC_ADMIN QUOTA UNLIMITED ON USERS;
```

The table and its indexes were moved out of `SYSTEM` into `USERS`:

```
ALTER TABLE CDC_ADMIN.CUSTOMERS
MOVE TABLESPACE USERS;
```

The existing indexes were rebuilt:

```
ALTER INDEX SYS_C008655 REBUILD TABLESPACE USERS;

ALTER INDEX SYS_C008656 REBUILD TABLESPACE USERS;
```

The index names above are the names observed in this environment. In a new database, identify the actual index names before rebuilding them.

4.2 Insert initial data

Three sample records were inserted and committed:

```
INSERT INTO CDC_ADMIN.CUSTOMERS
    (CUSTOMER_ID, FIRST_NAME, LAST_NAME, EMAIL, PHONE)
VALUES
    (1, 'Ahmed', 'Ali', 'ahmed@example.com', '01012345678');

INSERT INTO CDC_ADMIN.CUSTOMERS
    (CUSTOMER_ID, FIRST_NAME, LAST_NAME, EMAIL, PHONE)
VALUES
    (2, 'Mona', 'Hassan', 'mona@example.com', '01098765432');

INSERT INTO CDC_ADMIN.CUSTOMERS
    (CUSTOMER_ID, FIRST_NAME, LAST_NAME, EMAIL, PHONE)
VALUES
    (3, 'Omar', 'Mahmoud', 'omar@example.com', '01122334455');

COMMIT;
```

The rows were queried successfully after the commit.

These initial records will be useful when testing the connector's initial snapshot.

---

5. Enable table-level supplemental logging

Minimum supplemental logging is a database-level prerequisite. We also enabled all-column supplemental logging for the specific source table.

First, we connected as an administrative user and switched to `CDCDB`:

```
ALTER SESSION SET CONTAINER = CDCDB;
```

Then we executed:

```
ALTER TABLE CDC_ADMIN.CUSTOMERS
ADD SUPPLEMENTAL LOG DATA (ALL) COLUMNS;
```

Oracle returned:

```
Table CDC_ADMIN.CUSTOMERS altered.
```

5.1 Verify table-level supplemental logging

We verified the log group with:

```
SELECT LOG_GROUP_NAME, ALWAYS, LOG_GROUP_TYPE
FROM DBA_LOG_GROUPS
WHERE OWNER = 'CDC_ADMIN'
  AND TABLE_NAME = 'CUSTOMERS';
```

Observed result:

```
LOG_GROUP_NAME   ALWAYS   LOG_GROUP_TYPE
--------------   -------  ----------------------
SYS_C008657      ALWAYS   ALL COLUMN LOGGING
```

> **Result:** Oracle created a supplemental log group for `CUSTOMERS` with all-column logging.

This provides additional row-identification information in redo records. It also increases redo volume, so the scope of supplemental logging should be considered when applying the configuration to larger databases.

---

6. Create the Debezium database user

Debezium needs a dedicated Oracle account with permissions to access the required database metadata and use LogMiner.

We created a common user from `CDB$ROOT`:

```
CREATE USER C##DBZUSER
IDENTIFIED BY "<REPLACE_WITH_A_SECURE_PASSWORD>"
CONTAINER = ALL;
```

The account uses the `C##` prefix because it is an Oracle common user.

A common user can exist across the CDB and its PDBs, subject to the applicable Oracle configuration and privileges.

6.1 Verify the user

From `CDB$ROOT`, we ran:

```
SELECT USERNAME, COMMON, DEFAULT_TABLESPACE
FROM DBA_USERS
WHERE USERNAME = 'C##DBZUSER';
```

Observed result:

```
USERNAME      COMMON   DEFAULT_TABLESPACE
------------  -------  ------------------
C##DBZUSER    YES      LOGMINER_TBS
```

This confirmed that the account is a common user and that its default tablespace was configured successfully.

---

7. Grant the required database privileges

The following grants were executed from `CDB$ROOT` using a SYSDBA connection. They were applied with `CONTAINER = ALL`.

> **Important:** This is a record of the grants executed in this lab, not a claim that every grant is mandatory for every Debezium release or Oracle configuration. Before using this setup elsewhere, compare the permissions against the documentation for the exact Debezium Oracle connector version and its selected mining strategy. Some of these permissions are broad and should not be copied into production without DBA/security review.

7.1 Session and container access

```
GRANT CREATE SESSION, SET CONTAINER
TO C##DBZUSER CONTAINER = ALL;
```

Purpose:

- `CREATE SESSION` allows the user to connect to Oracle.
- `SET CONTAINER` allows switching containers where permitted.

7.2 Database and table access

```
GRANT SELECT ON V_$DATABASE
TO C##DBZUSER CONTAINER = ALL;

GRANT FLASHBACK ANY TABLE
TO C##DBZUSER CONTAINER = ALL;

GRANT SELECT ANY TABLE
TO C##DBZUSER CONTAINER = ALL;

GRANT SELECT_CATALOG_ROLE
TO C##DBZUSER CONTAINER = ALL;

GRANT EXECUTE_CATALOG_ROLE
TO C##DBZUSER CONTAINER = ALL;

GRANT LOGMINING
TO C##DBZUSER CONTAINER = ALL;

GRANT SELECT ANY TRANSACTION
TO C##DBZUSER CONTAINER = ALL;
```

These grants provide database metadata access, table-reading capabilities, and transaction-log mining permissions.

7.3 Object creation and table locking

```
GRANT CREATE TABLE
TO C##DBZUSER CONTAINER = ALL;

GRANT LOCK ANY TABLE
TO C##DBZUSER CONTAINER = ALL;

GRANT CREATE SEQUENCE
TO C##DBZUSER CONTAINER = ALL;
```

These privileges were granted during this lab setup. Whether all of them are needed depends on the connector version and configuration.

7.4 LogMiner package execution

```
GRANT EXECUTE ON DBMS_LOGMNR
TO C##DBZUSER CONTAINER = ALL;

GRANT EXECUTE ON DBMS_LOGMNR_D
TO C##DBZUSER CONTAINER = ALL;
```

These grants allow execution of the specified Oracle LogMiner packages.

7.5 Log and transaction metadata views

The following object-level grants were also executed:

```
GRANT SELECT ON V_$LOG
TO C##DBZUSER CONTAINER = ALL;

GRANT SELECT ON V_$LOG_HISTORY
TO C##DBZUSER CONTAINER = ALL;

GRANT SELECT ON V_$LOGMNR_LOGS
TO C##DBZUSER CONTAINER = ALL;

GRANT SELECT ON V_$LOGMNR_CONTENTS
TO C##DBZUSER CONTAINER = ALL;

GRANT SELECT ON V_$LOGMNR_PARAMETERS
```

```
TO C##DBZUSER CONTAINER = ALL;

GRANT SELECT ON V_$LOGFILE
TO C##DBZUSER CONTAINER = ALL;

GRANT SELECT ON V_$ARCHIVED_LOG
TO C##DBZUSER CONTAINER = ALL;

GRANT SELECT ON V_$ARCHIVE_DEST_STATUS
TO C##DBZUSER CONTAINER = ALL;

GRANT SELECT ON V_$TRANSACTION
TO C##DBZUSER CONTAINER = ALL;

GRANT SELECT ON V_$MYSTAT
TO C##DBZUSER CONTAINER = ALL;

GRANT SELECT ON V_$STATNAME
TO C##DBZUSER CONTAINER = ALL;
```

Each command returned `Grant succeeded`.

The permissions provide access to various online redo log, archived log, LogMiner, and transaction metadata views.

---

8. Configure the LogMiner tablespace

A dedicated tablespace named `LOGMINER_TBS` was created to provide storage for the connector's internal LogMiner-related objects.

8.1 Create it in `CDB$ROOT`

We connected as SYSDBA and confirmed the current container was `CDB$ROOT`.

```
CREATE TABLESPACE LOGMINER_TBS
    DATAFILE SIZE 25M
    AUTOEXTEND ON
    NEXT 10M
    MAXSIZE UNLIMITED;
```

Oracle returned:

```
TABLESPACE LOGMINER_TBS created.
```

8.2 Create it in `CDCDB`

We switched to the source PDB:

```
ALTER SESSION SET CONTAINER = CDCDB;
```

Then created the tablespace:

```
CREATE TABLESPACE LOGMINER_TBS
    DATAFILE SIZE 25M
    AUTOEXTEND ON
    NEXT 10M
    MAXSIZE UNLIMITED;
```

Oracle returned:

```
TABLESPACE LOGMINER_TBS created.
```

8.3 Create it in `FREEPDB1`

When setting the common user's default tablespace across all containers, Oracle returned:

```
ORA-65048: error encountered when processing the current DDL statement in pluggable database
    FREEPDB1
ORA-00959: tablespace 'LOGMINER_TBS' does not exist
```

The command failed because `LOGMINER_TBS` did not yet exist in `FREEPDB1`.

We switched to that PDB:

```
ALTER SESSION SET CONTAINER = FREEPDB1;
```

Then created the tablespace:

```
CREATE TABLESPACE LOGMINER_TBS
    DATAFILE SIZE 25M
    AUTOEXTEND ON
    NEXT 10M
    MAXSIZE UNLIMITED;
```

Oracle returned:

```
TABLESPACE LOGMINER_TBS created.
```

This additional tablespace was created to allow the common-user DDL using `CONTAINER = ALL` to succeed in every container where it was applied. It is not necessarily a universal requirement to create this tablespace in every PDB for every Debezium setup.

8.4 Set the default tablespace for the common user

We returned to the root:

```
ALTER SESSION SET CONTAINER = CDB$ROOT;
```

Then executed:

```
ALTER USER C##DBZUSER
DEFAULT TABLESPACE LOGMINER_TBS
CONTAINER = ALL;
```

Oracle returned:

```
User C##DBZUSER altered.
```

8.5 Configure the quota

We then executed:

```
ALTER USER C##DBZUSER
QUOTA UNLIMITED ON LOGMINER_TBS
CONTAINER = ALL;
```

Oracle returned:

```
User C##DBZUSER altered.
```

This sets the user's tablespace quota to unlimited relative to the tablespace's available capacity. It does not remove the database's physical storage limits.

---

9. Verification checklist

The following checks were completed during this setup.

Check	Result
Oracle is running in ARCHIVELOG mode	**Verified**

Check	Result
Minimum supplemental logging	**Enabled and verified as** `YES`
Source PDB	`CDCDB`
Source schema and table	`CDC_ADMIN.CUSTOMERS` created
Initial sample records	**Three rows inserted and committed**
Table-level supplemental logging	**Enabled and verified**
Common Debezium user	`C##DBZUSER` created
User is common	Verified as `YES`
Default tablespace	Verified as `LOGMINER_TBS`
Tablespace quota	Configured successfully
Privilege grant commands	All commands listed above returned success

Verify the current container

```
SELECT SYS_CONTEXT('USERENV', 'CON_NAME') AS CONTAINER_NAME
FROM DUAL;
```

Verify ARCHIVELOG and minimum supplemental logging

Run as a suitably privileged user:

```
SELECT LOG_MODE, SUPPLEMENTAL_LOG_DATA_MIN
FROM V$DATABASE;
```

Expected values:

```
ARCHIVELOG
YES
```

Verify the source table's log group

Connect to `CDCDB` and run:

```
SELECT LOG_GROUP_NAME, ALWAYS, LOG_GROUP_TYPE
FROM DBA_LOG_GROUPS
WHERE OWNER = 'CDC_ADMIN'
  AND TABLE_NAME = 'CUSTOMERS';
```

Expected: a log group with `ALL COLUMN LOGGING`.

---

10. What remains to be done

Oracle source preparation is not the same as configuring the Debezium connector. The following stages remain.

1. **Review permissions against the exact Debezium release.** Confirm the required grants and any additional privileges for the selected connector version and LogMiner configuration. Remove unnecessary broad privileges before any production use.
2. **Validate database-user connectivity.** Test a connection as `C##DBZUSER` using the appropriate Oracle service and container configuration.
3. **Start Kafka and Kafka Connect.** Confirm the broker and Connect REST API are available.

4. **Install or select the Debezium Oracle connector plugin.** Confirm that Kafka Connect can discover the plugin.
5. **Configure the Oracle source connector.** Set the source connection, PDB, schema/table filters, topic naming, and LogMiner options.
6. **Register the connector.** Submit its configuration to the Kafka Connect REST API.
7. **Validate the initial snapshot.** Confirm that the existing `CUSTOMERS` rows are emitted.
8. **Test CDC.** Insert, update, and delete sample records and verify that the corresponding events arrive in Kafka.
9. **Configure the destination connector or consumer.** Deliver the events to the target database and validate the end-to-end pipeline.

11. Official documentation

- Debezium Oracle connector: https://debezium.io/documentation/reference/stable/connectors/oracle.html
- Oracle Database documentation: https://docs.oracle.com/en/database/
- Oracle SQL Language Reference: https://docs.oracle.com/en/database/oracle/oracle-database/23/sqlrf/

Final status: Oracle source preparation is substantially complete for this local proof of concept. Connector-specific privileges and connectivity still need validation against the chosen Debezium release before the source connector is registered and tested.